

Lectures – 24, 25, 26

DIJKSTRA'S SHORTEST PATH ALGORITHM

G is a weighted, directed graph $G = (V, E)$ with weight function w , which does not allow negative edge weights.

The weight $w(p)$ of a path, the shortest path weight $\delta(u, v)$ for the pair of vertices u and v , and shortest path are defined as before.

The property of optimal sub-structure holds, and so greedy strategy works.

Predecessor subgraph, specified source vertex s and shortest path tree rooted at s is exactly as defined before.

Relaxation – as before, a key step.

Here $d[v]$ is an attribute of every vertex v in V , which is an upper bound on the weight of the shortest path from s to v . We call it **shortest path estimate**. It is initialized to ∞ for all the vertices except s , for which it is set to 0.

```
RELAX( $u, v, w$ )  
  if  $d[v] > d[u] + w(u, v)$   
     $d[v] = d[u] + w(u, v)$ ; //  $d[v]$  and  $pred[v]$   
     $pred[v] = u$ ;          // are updated
```

The predecessor $pred[v]$ or $\pi[v]$ of every vertex v in V is initialized to NULL. Recall the almost identical step in earlier algorithms.

The following is a straightforward function for initialization:

```
INITIALIZE-SINGLE-SOURCE( $G, s$ )  
  for each vertex  $v$  in  $V[G]$   
     $d[v] = \infty$ ;  
     $pred[v] = NIL$ ;  
   $d[s] = 0$ ;
```

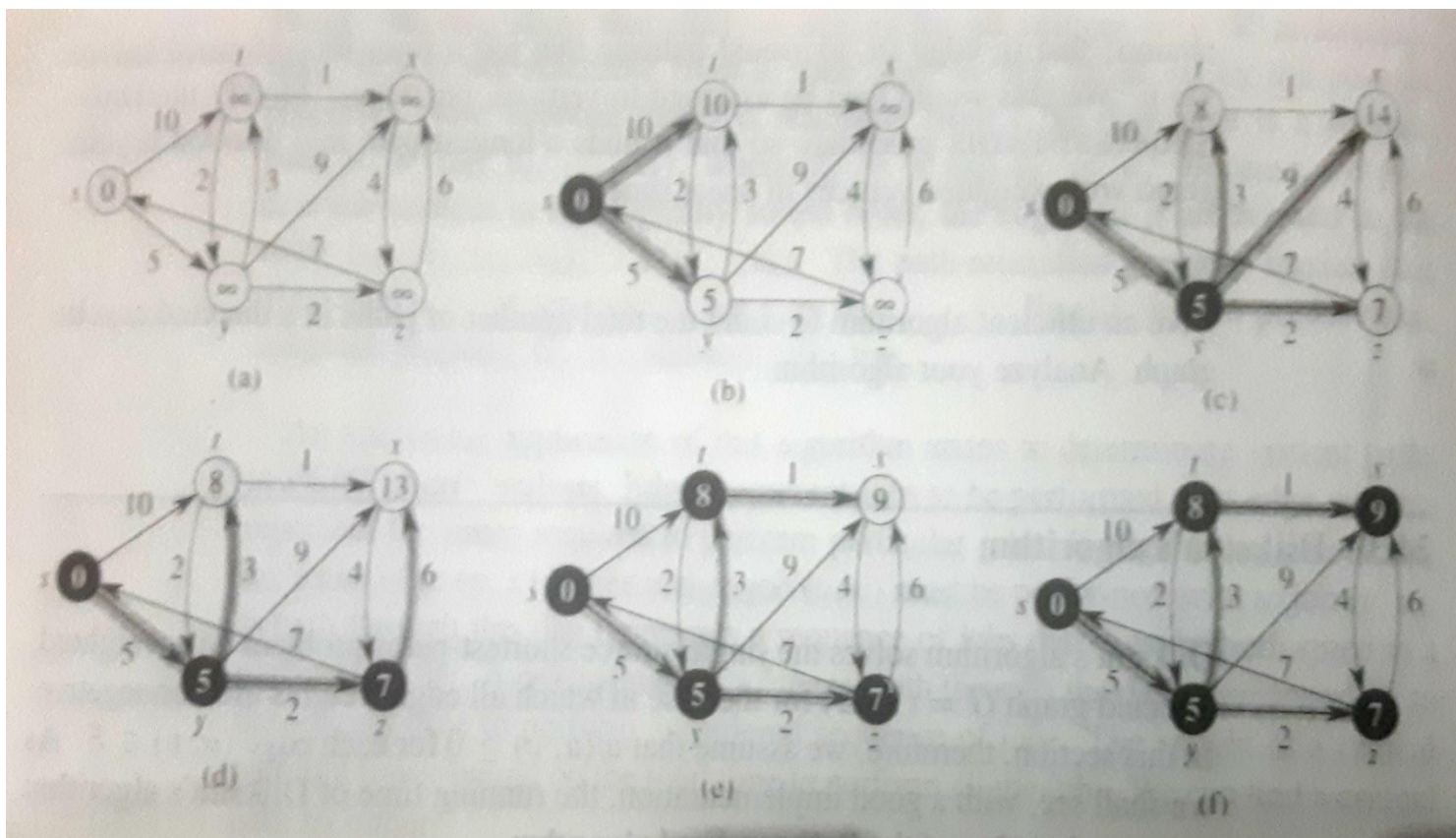
Now we consider Dijkstra's algorithm:

```

DIJKSTRA (G,w,s)
// Initialization
  INITIALIZE-SINGLE-SOURCE (G,s) ;
  S = NULL;           // S is a set of processed vertices
  Q = V[G];           // Q is a min-priority queue
// Main loop
  while Q <> NULL
    u = EXTRACT-MIN (Q) ;
    S = S ∪ {u};
    for each v in ADJ[u]
      RELAX (u,v,w) ;

```

Example [from CLRS]:



Running time:

$O(E \lg V)$ if the min-priority queue is implemented using binary min-heap.

$O(V \lg V + E)$ if the min-priority queue is implemented using Fibonacci heap.

Question: What is the difference between a "sparse" and a "dense" graph?

ALL-PAIRS SHORTEST PATHS

Common example: Table of road distances between every pair of cities.

Important to note: Road network can be assumed to be **planar**.

Suppose we run Dijkstra's shortest path algorithm N times, for N nodes, making each node the source node one by one. Of course, for this we must assume non-negative edge weights.

Then the all-pairs shortest paths can be computed in $O(VE \lg V)$ time. This is alright if the graph is sparse; improvement is possible with Fibonacci heap.

Matrix methods are useful in solving the all-pairs shortest paths problem. Therefore we shall use the adjacency matrix representation of graphs.

Element w_{ij} in the adjacency matrix W is given by:

$w_{ij} = 0$, if $i = j$,

$w_{ij} = \infty$, if there is no edge in G from vertex i to vertex j ,

$w_{ij} = \text{edge weight}$, if there is an edge in G from vertex i to vertex j .

Note: We assume that there are no negative edge weights.

Predecessor subgraphs

Recall that, when you consider the shortest paths to every other node from a given node i , you get a predecessor subgraph tree rooted at that node.

This tree helps us to re-construct the shortest path.

So now, considering all N possible root nodes in turn, we will have a total of N predecessor subgraph trees, the i^{th} tree being rooted at node i , for $i = 1, 2 \dots N$.

The information about all these N predecessor subgraph trees can be stored in an $N \times N$ matrix, compiled while computing the shortest paths.

Some more details are given in [CLRS].

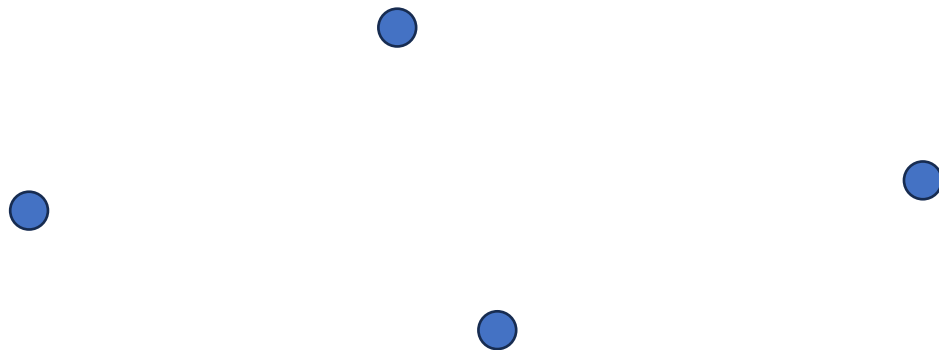
Brief review of usual matrix multiplication

Say $n \times n$ matrices A and B are multiplied to give $n \times n$ matrix C . That is, $C = AB$.

$$c_{ij} = \sum_{k=1, n} (a_{ik} * b_{kj})$$

Note the arithmetic operations involved: n **multiplications** and n **additions**, in calculating each c_{ij} , $i, j = 1, 2, \dots, n$. Assume that multiplications dominate. Then it follows that the straightforward matrix multiplication takes $\Theta(n^3)$ time.

Now think of k as being an intermediate vertex in G on a path from vertex i to vertex j . And consider two such paths.



In the diagram, there are two such paths: p_1 and p_2 . Vertex k' is intermediate on path p_1 , and vertex k'' is intermediate on path p_2 .

Length of p_1 = length of $p_1\langle i, k' \rangle$ + length of $p_1\langle k', j \rangle$

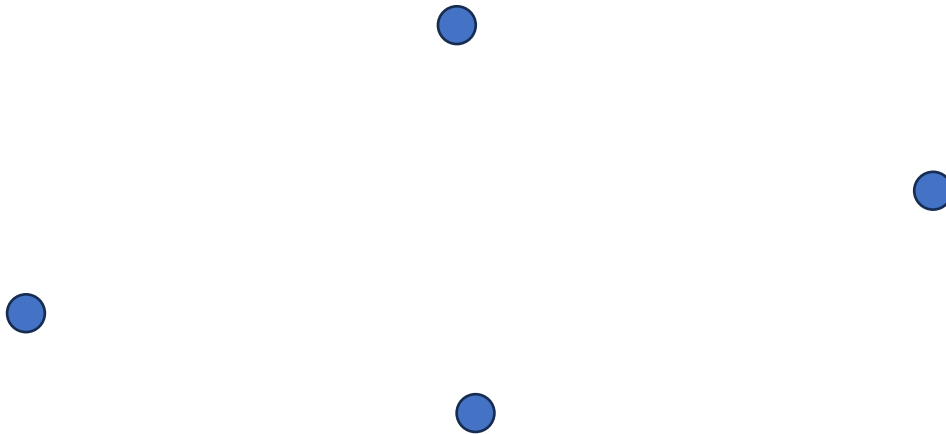
Length of p_2 = length of $p_2\langle i, k'' \rangle$ + length of $p_2\langle k'', j \rangle$

Shorter path length from i to j = **min** (length of p_1 , length of p_2)

The two key operations here are: **addition** and **minimum**.

It was discovered early on that we can apply the basic concept of matrix multiplication to our problem! The trick is to use **addition** and **minimum** in place of the usual two arithmetic operations.

Simple example – to be solved manually



FLOYD-WARSHALL ALGORITHM FOR ALL-PAIRS SHORTEST PATHS

[Highly simplified from CLRS]

Consider any two vertices i and j in the weighted, directed graph G . Now consider the distance $d_{ij}^{(k)}$ from vertex i to vertex j defined as follows:

$d_{ij}^{(k)}$ = distance of the shortest path from vertex i to vertex j
subject to the condition that **no intermediate**
vertex on the path is numbered $\geq k$

Recursive solution

A recursive solution to the problem is based on the following two observations:

$d_{i,j}^{(0)} = W_{i,j}$ because when $k = 0$, there are no intermediate vertices on the path from vertex i to vertex j .

$$d_{i,j}^{(k)} = \min(d_{i,j}^{(k-1)}, d_{i,k}^{(k-1)} + d_{k,j}^{(k-1)}) \quad \text{for } k = 1, 2, \dots, n$$

This is because the shortest path of the type considered, from vertex i to vertex j , **MUST** satisfy one of these two conditions:

- (1) it does NOT contain intermediate vertex k , OR
- (2) it contains exactly one occurrence of intermediate vertex k .

[This path, by definition, does not contain vertices numbered higher than k .]

Floyd-Warshall algorithm is based on the above two observations.

FLOYD-WARSHALL (W)

```

n = rows[W];
D(0) = W;
for k = 1 to n
  for i = 1 to n
    for j = 1 to n
      D(i, j)(k) = min( D(i, j)(k-1),
                      D(i, k)(k-1) + D(k, j)(k-1) );
return D(n);

```

Note: Operation of the function **min(x, y)** here is very similar to what is done within the function **RELAX** which we studied earlier.

The predecessor subgraphs **s** are also constructed in a similar way.
[Shown later in CLRS].

Running time analysis: Straightforward. $\Theta(n^3)$.

Example: See below.

First we will draw the graph based on the matrix $W = D^{(0)}$.

$$D^{(0)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

$$\Pi^{(0)} = \begin{pmatrix} \text{NIL} & 1 & 1 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & \text{NIL} & \text{NIL} \\ 4 & \text{NIL} & 4 & \text{NIL} & \text{NIL} \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix}$$

$$D^{(1)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \textcircled{5} & -5 & 0 & \textcircled{-2} \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

$$\Pi^{(1)} = \begin{pmatrix} \text{NIL} & 1 & 1 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & \text{NIL} & \text{NIL} \\ 4 & \textcircled{1} & 4 & \text{NIL} & \textcircled{1} \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix}$$

$$D^{(2)} = \begin{pmatrix} 0 & 3 & 8 & \textcircled{4} & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \textcircled{5} & \textcircled{11} \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

$$\Pi^{(2)} = \begin{pmatrix} \text{NIL} & 1 & 1 & 2 & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & 2 & 2 \\ 4 & 1 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix}$$

$$D^{(3)} = \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

$$\Pi^{(3)} = \begin{pmatrix} \text{NIL} & 1 & 1 & 2 & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & 2 & 2 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix}$$